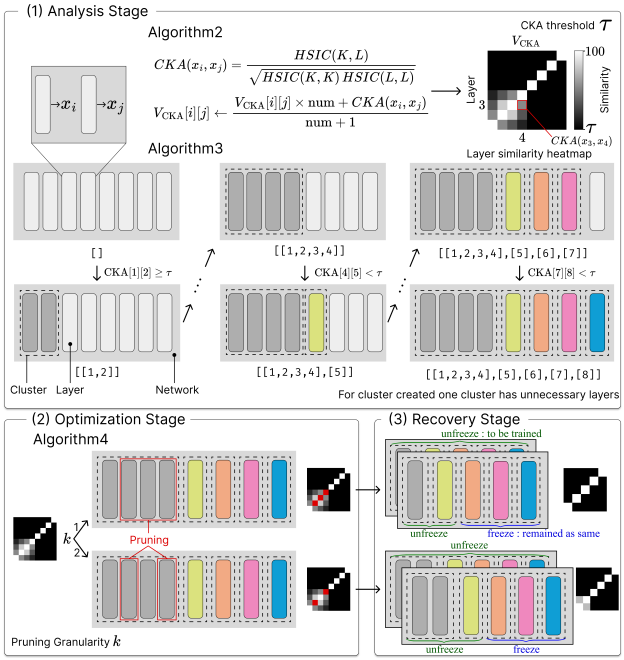


Figure 2: Three Phases of MPruner: (1) Analysis Stage with CKA Similarity Score, (2) Optimization Stage with Multi-Layer Cluster Pruning, and (3) Recovery Stage with Retraining.

model is examined to provide multi-layer cluster-wise information. The second phase is the **(2) optimization stage**, where unnecessary and redundant layers with minimal impact on the model's functionality are removed. The third phase is the **(3) recovery stage**, in which the model's performance is restored through retraining.

The overall mechanism is illustrated in Algorithm 1. It takes five inputs: the model $\mathcal{M}$, the dataset $\mathcal{D}$, the hook position $\mathcal{L}$ to specify where to calculate the CKA-based mutual information metrics, the CKA threshold τ to determine the multi-layer clustering, the accuracy drop threshold γ to determine when the algorithm should stop, and the freeze flag b to indicate whether retraining should involve the entire network or focus only on the pruned parts. The algorithm sets the pruning granularity to 1 (line 4), meaning that all layers except one in redundant layer clusters will be removed. If set to 2, it prunes layers by skipping one after pruning one, effectively removing half of the redundant layers.

During each iteration, the algorithm first calculates and records the CKA values (line 6). If no layer clusters are found to perform similar functionalities in the model, the algorithm terminates (line 7). Otherwise, it invokes the pruner to remove redundant layers inside the multi-layer cluster according to the specified pruning granularity (line 8). The pruner then returns the pruned and untrained network $\mathcal{M}_{p,u}$ and the layers ls_f that should be frozen if retraining is required. After retraining the pruned network, the algorithm calculates the accuracy of the updated version (lines 10 and 11), then adjusts either the model or the pruning granularity

Algorithm 1: `MPruner`

Require: model $\mathcal{M}$, dataset $\mathcal{D}$, hook positions $\mathcal{L}$, CKA threshold τ, accuracy drop threshold γ, freeze flag b
Ensure: pruned and retrained model
1: $\mathcal{M}^* \leftarrow \mathcal{M}$
2: $\mathcal{C} \leftarrow []$
3: $acc_o, acc_{pr} \leftarrow$ **GetAccuracy**$(\mathcal{M}, \mathcal{D})$
4: $k \leftarrow 1$ (default pruning granularity)
5: **while** $k \leq 3$ **do**
6: $\quad\mathcal{C} \leftarrow$ **GetCandidates**$(\mathcal{M}, \mathcal{L}, \tau, \mathcal{D})$
7: $\quad$**if** $\forall$ el $(\in \mathcal{C})$. length(el) == 1 **then break**
8: $\quad(\mathcal{M}_{p,u}, ls_f) \leftarrow$ **Pruner**$(\mathcal{M}, \mathcal{C}, k)$
9: $\quad\mathcal{M}_{f,u} \leftarrow$ **Freezer**$(\mathcal{M}_{p,u}, ls_f, \mathcal{L}, b)$
10: $\quad\mathcal{M}^* \leftarrow$ **Trainer**$(\mathcal{M}_{f,u})$
11: $\quad acc_{pr} \leftarrow$ **GetAccuracy**$(\mathcal{M}^*, \mathcal{D})$
12: $\quad$**if** $acc_o - acc_{pr} \leq \gamma$ **then** $\mathcal{M} \leftarrow \mathcal{M}^*$ **else** $k \leftarrow k+1$
13: **end while**
14: **return** $\mathcal{M}$

Algorithm 2: `Calculate the CKA matrix`

Require: $\mathcal{M}, \mathcal{L}, \mathcal{D}$
Ensure: $\mathcal{V}_{CKA}$
1: $\mathcal{V}_{CKA}[][] \leftarrow \{0\}$
2: $\mathcal{D}_{seed}[] \leftarrow$ **GetSeed**$(\mathcal{D})$
3: **for** $(idx \in$ **len**$(\mathcal{D}_{seed}))$ **do**
4: $\quad i \leftarrow \mathcal{L}[0]$
5: $\quad output[] \leftarrow$**GetOutputs**$(\mathcal{M}, \mathcal{L}, \mathcal{D}_{seed}[idx])$
6: $\quad$**for** $j \in \mathcal{L}[1:]$) **do**
7: $\quad\quad cka \leftarrow$**GetCKA**$(output[i], output[j])$
8: $\quad\quad\mathcal{V}_{CKA}[i][j] \leftarrow (\mathcal{V}_{CKA}[i][j] * idx + cka)/(idx+1)$
9: $\quad\quad i \leftarrow j$
10: $\quad$**end for**
11: **end for**
12: **return** $\mathcal{V}_{CKA}$

based on the observed accuracy drop (line 12).

Instead of specifying a sparsity level to prune the model parameters to a certain extent, the algorithm terminates when no more layers can be pruned or when the pruning granularity becomes too coarse. These conditions indicate that the network either has no layers left to prune or is no longer suitable for further pruning.

Two key functions in the algorithm are **GetCandidates** and **Pruner**. The **GetCandidates** function identifies layers within the network that perform similar functions. Algorithm 3 outlines the essential steps for identifying these similarities, utilizing Algorithm 2, which calculates the CKA values for all layers in the model and records them in the CKA matrix ($\mathcal{V}_{CKA}$). Once the CKA matrix is generated, pruning candidates are selected based on CKA value similarity: if the output vectors of two or more adjacent layers fall within a specified threshold, they are grouped together as a pruning candidate cluster ($\mathcal{C}$).

The **Pruner**, as detailed in Algorithm 4, reduces the size of the target model based on the results from **GetCandidates** and the specified pruning granularity. If a layer cluster identified in the candidate set consists of two or more layers

Algorithm 3: `Get Pruning Candidates`

Require: $\mathcal{M}, \mathcal{L}, \tau, \mathcal{D}$
Ensure: pruning candidate clusters $\mathcal{C}$

$\mathcal{V}_{CKA} \leftarrow$ **GetCKAMatrix**$(\mathcal{M}, \mathcal{L}, \mathcal{D})$
$\mathcal{C} \leftarrow []$
$i \leftarrow \mathcal{L}[0]$
$cluster \leftarrow \{i\}$
for $j \in \mathcal{L}[1:]$ **do**
  if $\mathcal{V}_{CKA}[i][j] \geq \tau$ **then** $cluster \cup \{j\}$
  else append $cluster$ **to** $\mathcal{C}$
  $cluster \leftarrow \{j\}$
  $i \leftarrow j$
end for
return $\mathcal{C}$

Algorithm 4: `Pruning with Clustering Info.`

Require: $\mathcal{M}, \mathcal{C}, k$
Ensure: Pruned model $\mathcal{M}^*$, non-freeze layers ls_f

$ls_f \leftarrow \{\}$
$\mathcal{M}^* \leftarrow \mathcal{M}$
for $cluster \in \mathcal{C}[:-1]$ **do**
  for $l_{idx} \in cluster[1:k][:-1]$ **do**
    if $\text{dim}(\text{Out}_{pre_layer}(l_{idx}) == \text{dim}(\text{In}_{next_layer}(l_{idx})$
    then $\mathcal{M}^* \leftarrow$ **DeleteLayer**$(\mathcal{M}^*, l_{idx})$
    else $ls_f \leftarrow ls_f \cup \{l_{idx} - 1, l_{idx}\}$
  end for
end for
return $\mathcal{M}^*, ls_f$

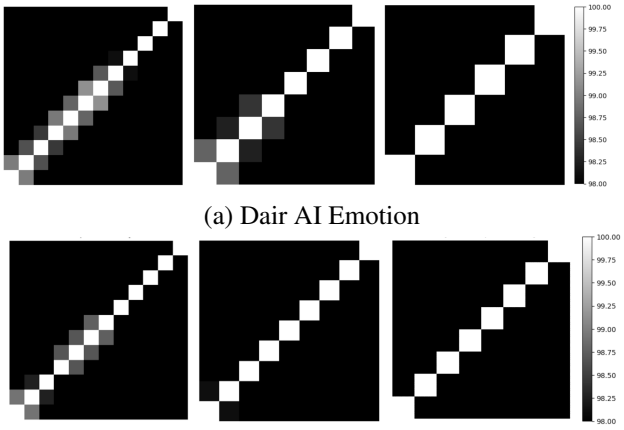
and the pruning granularity is set to 1, our approach initially attempts to remove all layers in the cluster except the first one, based on the evidence that these layers perform similar roles within the model. For instance, if n adjacent layers form a module ($p_i = l_1, \ldots, l_n$) and the output vectors from l_1 to l_n fall within a similarity threshold τ or the given input values, merging these layers is expected to have minimal impact on the model's performance.

However, we observe that excessive pruning sometimes leads to a noticeable drop in accuracy, particularly for CNN models. We believe this issue arises from having too many layers within a single cluster. To address this, we offer the option to adjust the pruning granularity to values other than 1. If the granularity is set to 2, the algorithm will attempt to remove every other layer in the cluster, effectively pruning the even-numbered layers.

As defined in Algorithm 1, our algorithm adjusts the pruning ratio if the pruned model experiences a significant accuracy drop due to the reduction of multiple layers and fails to meet the accuracy threshold. We have found that such adjustments are generally unnecessary for transformer models. However, CNN models (such as ResNet (He et al. 2016)) may require this step to mitigate accuracy loss.

Experiments

We conducted a range of evaluations by varying the inputs to MPruner. The primary goal of our experiments was not to



(a) Dair AI Emotion

(b) Yahoo Answers Topics

Figure 3: Clustering Heatmaps for Bert.

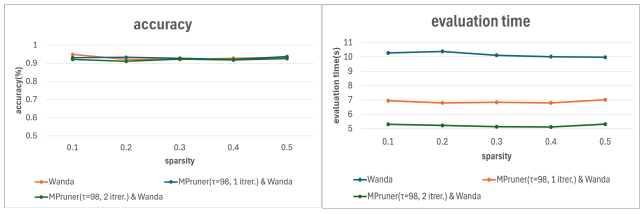


Figure 4: Result of Wanda (Sun et al. 2024), and Integration of Wanda and MPruner.

argue that our method is superior to previous pruning techniques, but to confirm whether the multi-layer cluster pruning described in Section can achieve the optimal size, as supported by mathematical evidence measured by our CKA similarity score. All code and models were implemented in PyTorch. Due to the diversity of our experiments, we utilized multiple GPUs for analysis, optimization, recovery, and performance validation both before and after pruning. Specific details of the experimental setup are provided. As part of the inputs for MPruner, we set the CKA threshold (τ) to 95%, 98%, and/or 99%. The retraining process was conducted with a learning rate of 0.00005, a batch size of 32, and for 3 epochs. To ensure a thorough comparison, we report all intermediate results, not just the final input and output models of our top-level pruning algorithm 1.

Overall, we observed that our method effectively optimizes the size of both CNN and Transformer models with high scalability. The analysis stage operates in linear time, efficiently determining global multi-layer cluster-wise information regardless of the network's depth or width. Additionally, our approach demonstrated strong effectiveness for Transformer models, including both encoder-only and encoder-decoder architectures. We also found that our method can be seamlessly integrated with state-of-the-art magnitude pruning techniques (Sun et al. 2024) without causing any accuracy loss in Transformer models.

	Method	Performance(%)	Evaluation time(s)	Training time(s)	Parameter(M)	Num. of Encoders
Dair AI Emotion	Baseline	0.92	14.8684	1579.8736	109.4869	12
	τ = 99%, unfreeze, 1st iter.	0.92	8.6455	1306.8720	95.3111	10
	τ = 98%, unfreeze, 1st iter.	0.93 (+0.01)	7.0948	1062.2365	81.1354	8
	τ = 98%, unfreeze, 2nd iter.	0.93 (+0.01)	5.5214	796.2302	66.9596	6
Yahoo Answers Topics	Baseline	0.70	101.9891	41325.0308	109.4869	12
	τ = 99%, unfreeze, 1st iter.	0.70	101.9891	N.A.	109.4869	12
	τ = 98%, unfreeze, 1st iter.	0.71 (+0.01)	79.4448	31593.5162	88.2263	8

Table 1: Bert Compression.

Transformer models

BERT-based We performed two experiments with a BERT-based model. For small-scale data and label-related compression, we used a pretrained language emotion analysis model with six classes (Dair AI Emotion)[1]. This dataset was divided into 16,000 training samples, 1,000 validation samples for CKA analysis, and 1,000 evaluation samples. For large-scale data and label-related experiments, we used a pretrained news article topic classification model with ten classes (Yahoo Answers Topics)[2]. This dataset was split into 1,400,000 training samples, 30,000 validation samples for CKA analysis, and 30,000 evaluation samples. The experiments were conducted on two different setups: the Dair AI model was tested on an NVIDIA GeForce RTX 3070 Ti 8GB GPU with a 13th Gen Intel(R) Core(TM) i7-13700 processor and 16GB of RAM, while the Yahoo Answers Topics model was tested on an NVIDIA GeForce RTX 4090 24GB GPU with a 14th Gen Intel(R) Core(TM) i9-14900 processor and 64GB of RAM.

Table 1 presents the experimental results. For a straightforward comparison, we disabled the freezing option in our configuration. We also evaluated several aspects affected by $MPruner$: performance (accuracy), evaluation time (the total inference time on our validation dataset), and training time (the duration required for training with a batch size of 32 and 3 epochs). Our method shows promising results, effectively reducing the model size by up to 50% in terms of the number of encoders without compromising accuracy. Also, the final experimental results indicate that MPruner-had almost no effect on accuracy. Meanwhile, inference speed, measured as the total inference time on the validation dataset, improved by 62.04% and 22.10%, and training time was reduced by 49.60% and 23.55%, respectively. Although the compression ratio varies depending on the dataset size, our methodology reliably identifies layers that can be collapsed, ensuring minimal loss of accuracy with mathematical rigor, as depicted in Figure 3. The figure displays the clustering results for both experiments: from left to right, showing BERT-base, the results after a single pruning iteration, and the final compression outcome. The proposed methodology in this paper involves iterative clustering and pruning until no further clustering of two or more encoders is possible. This heatmap illustrates the gradual reduction in layer similarity, providing evidence that $MPruner$ prunes the target network conservatively but correctly. The pruning is performed up to the point where the model remains safe and continues to offer nearly identical functionalities before and after pruning.

[1]https://huggingface.co/datasets/dair-ai/emotion

[2]https://huggingface.co/datasets/yahoo_answers_topics

	Baseline	1st iter.	2nd iter.
Num. of encoder	12	9	7
Num. of decoder	12	5	3
Num. of parameter	222M	135M	109M
BLEU	0.2268	0.2120	0.2009
ROUGE-1 F1	0.5177	0.5027	0.4929
ROUGE-2 F1	0.3033	0.2880	0.2765
ROUGE-L F1	0.4812	0.4697	0.4601
ROUGE-Lsum F1	0.4812	0.4702	0.4601
METEOR	0.4894	0.4702	0.4590

Table 2: Compression on CodeT5

Potential for Integration with Other Tools MPruner is designed to identify the optimal pruning point that maintains mathematical guarantees, rather than pushing the network to its limits. Consequently, it does not focus solely on minimizing the number of parameters, layers, or memory usage. To validate the safety and effectiveness of our pruning results, we conduct stress tests and apply an established pruning technique (Sun et al. 2024) to both the original and pruned networks produced by MPruner. Our findings, illustrated in Figure 4, show that the accuracy of the pruned network is maintained at similar sparsity levels compared to the original model when pruned. Additionally, the reduced size of the network produced by MPruner enables its combination with Wanda, which further decreases inference time by eliminating additional weights compared to using Wanda alone. This empirically demonstrates that MPruner effectively removes unnecessary components while preserving network functionality and can enhance results when integrated with other pruning methods.

CodeT5 Another experiment evaluates the applicability and impact of our method on a natural language generation model, specifically the T5-base model using the SQuAD dataset[3]. The experiment was conducted on an NVIDIA RTX A6000 48GB GPU, an Intel(R) Xeon(R) W5-2455X processor, and 256 GB of RAM. Unlike previous benchmarks that used accuracy as the sole performance metric, we employed three well-known metrics—BLEU, ROUGE, and METEOR (Liu et al. 2016)—to assess the effectiveness of our approach. We set the CKA similarity threshold τ to

[3]https://huggingface.co/datasets/rajpurkar/squad